# The last step of index calculus: our linear algebra against CADO-NFS, and a port of its filter

Study `research/la_engines_20261014` · 2026-10-14/15 · host: 4 vCPU Intel Xeon @ 2.80 GHz cloud VM, Linux 6.18, shared, no core reservation (isolation L0) · CADO-NFS commit `692ecb7e` (2026-10-02)

## 1. Question and answer

**Is the linear-algebra step of our index calculus as fast as it can be, and is it holding back end-to-end solves?**

- **End to end it is not the bottleneck.** In real `ic logs` precomputes on Koblitz curves the linear algebra took 0.024–0.63 ms of 6.9–98 ms, 0.11–1.9% of the run (27 runs, three modes, `raw/e2e.jsonl`). The largest real relation matrix in the repository, the N73 rank-600 log table, needed 5.1 h of single-thread relation collection (57–70 M probes a relation, `experiments/koblitz-single-target-n73-20261003/README.md`); a solve of that size takes milliseconds here.
- **The solver was already fast; the filter was not.** On an identical matrix our block Wiedemann beats CADO-NFS `bwc` at every size measured (1.9× at 131,072 columns, more below). But CADO-NFS's filter (`purge` + `merge-dl`) left a 5–8× smaller matrix than ours, so its whole pipeline beat ours by about 6× at D = 262,144.
- **This change ports CADO-NFS's filter into `koblitz_sparse_la` and speeds it up.** Measured in one session (amendment 5), three seeds, D = 262,144: our pipeline went from 86.6 s to 8.35 s (**10.4×**, per-seed 9.99–11.06×), and against CADO-NFS on the same matrices ours now takes 7.83 s to CADO-NFS's 14.73 s (**1.93× faster**, per-seed 1.74–1.96×). Every run recovered every planted logarithm.
- **Limits.** Below about 4,096 columns the new filter is slower than the old one (0.40× at D = 1,024; both take milliseconds). The matrices are synthetic (m = 3 summands per relation, uniform columns, 62-bit modulus); the host is shared and unreserved, so wall times are descriptive and compared only within a session. No ECDLP speed-up follows: the linear algebra is under 2% of every real run.

## 2. Method

**Instrument.** `examples/la_engine_bench.rs` builds a deterministic synthetic relation system per (D, m, seed): rows of m = 3 summands at uniform columns with coefficients ±1 (or +1 for the CADO-NFS arms, whose relation format stores nonnegative valuations), drawn until every column occurs, then 32 more (1,049,656 rows at D = 262,144), with right-hand sides from a planted logarithm vector modulo $p = 2^{62} - 57$. Every arm's answer is compared with the planted vector (`verified`). The time reported is `solve_s`, from the relation rows to verified logarithms, in-process; for CADO-NFS the sum of its binaries' wall times plus the native right-hand-side assembly and reconstruction. Matrix generation and the CADO file export are input preparation and excluded on both sides.

**Arms.** Our engines: `IncrementalGauss` (dense, rows fed one at a time; the ecbench default), `StructuredGauss`, `solve_sparse_system` (filter, then block Wiedemann 4×4 or block Lanczos 4, then reconstruction), and block Wiedemann alone on a fixed core. CADO-NFS: `bwc.pl :complete` (GF(p), inhomogeneous, m = n = 4, `thr=2x2`), and its DL filter `purge -keep 160` → `merge-dl -target_excess 0` → `replay-dl` → `bwc`, reconstructed natively.

**Protocol.** `PROTOCOL.md` was frozen before data and five amendments (`PROTOCOL_AMENDMENT_1..5.md`) were each committed before their own data. Amendment 3 re-ran the 38 runs that overlapped builds or diagnostics (`raw/contended_runs.jsonl`); the figures use the re-runs. Raw records: `raw/*.jsonl`; CADO logs: `raw/cado_logs/`, `raw/cado_full_logs/`; figures from `scripts/plots.py` (layout only), medians and ranges in `summary.json`.

# 3. Results

## 3.1. Every solver against matrix size (frozen grid)

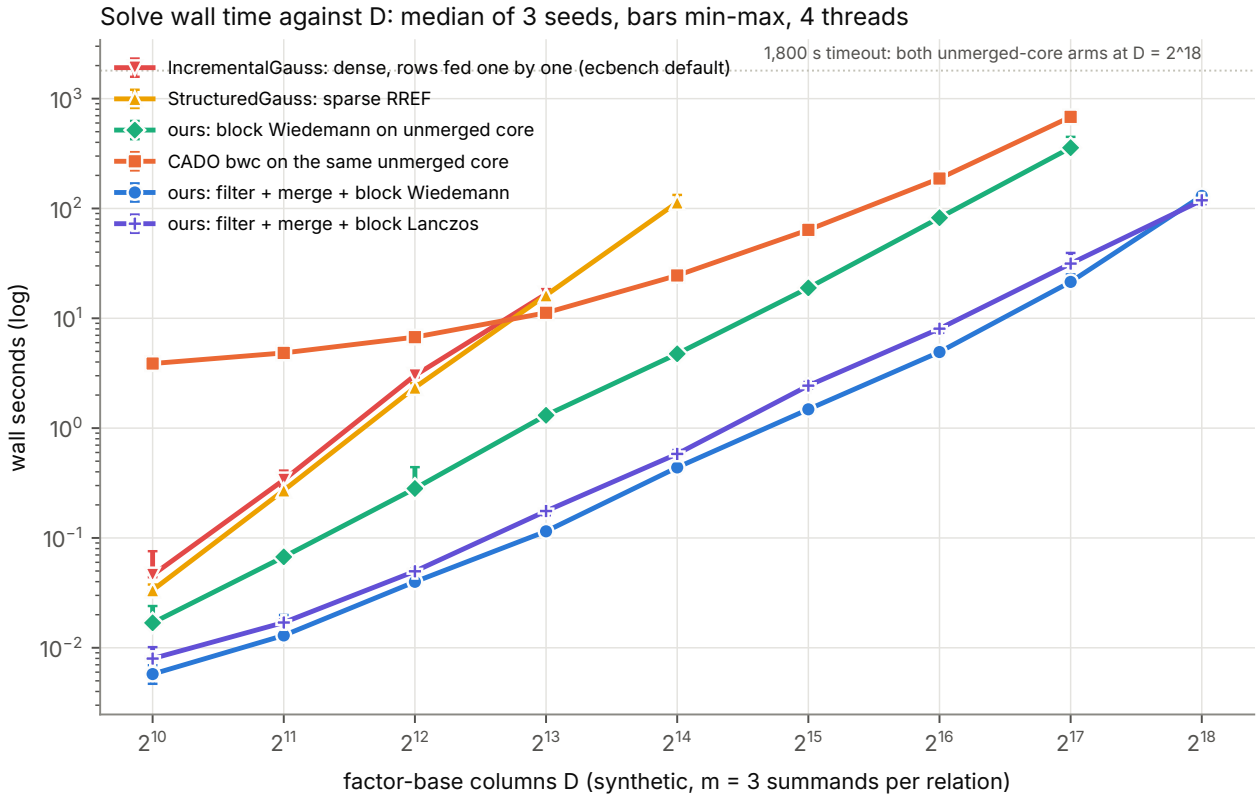



Figure 1: Frozen grid, ±1 coefficients, 4 threads, median of 3 seeds, bars min–max. Our block Wiedemann and CADO `bwc` on the unmerged core both hit the 1,800 s timeout at D = $2^{18}$.

- `IncrementalGauss` (ecbench's default) grows by 6.3–8.9× per doubling: 0.046 s at D = 1,024, 3.02 s at 4,096, 19.1 s at 8,192. `StructuredGauss` fills in faster still: 113.5 s at 16,384 (690–736 s at m = 4).
- The filtered sparse solve (legacy filter) took 0.44 s at 16,384, 4.93 s at 65,536, 130 s at 262,144; block Lanczos was within 1.0–1.6× of block Wiedemann throughout (118.7 s at 262,144).
- Threads: our block Wiedemann on the 32,860-column core took 102.6–104.7 s on one thread against 81.2–83.4 s on four (1.25×); CADO `bwc` 224.9–229.4 s against 183.3–196.7 s.

### 3.2. Engine against engine on the same matrix

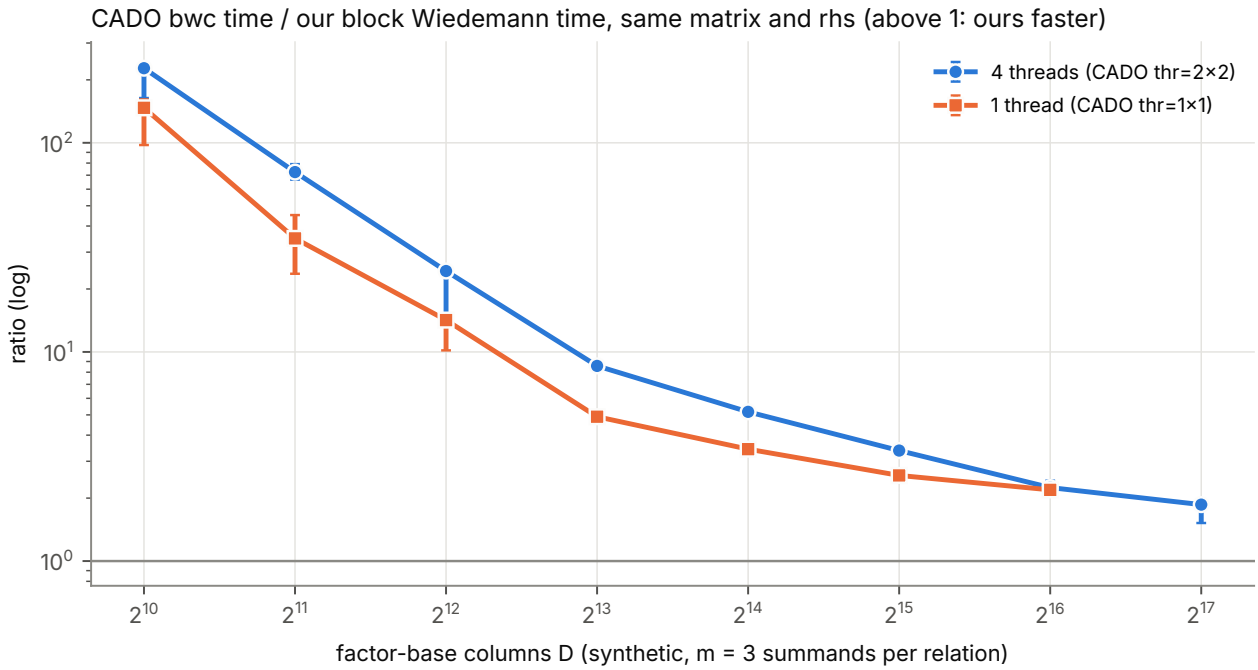



Figure 2: CADO `bwc` time over our block Wiedemann time on the identical unmerged core (singleton and clique removal only, so CADO's 32-bit coefficients can carry it), per (D, seed).

Ours is faster at every size: 227× at D = 1,024 (CADO's fixed cost: about 3–4 s for `prep`, `secure`, `krylov` start-up), 5.2× at 16,384, 2.25× at 65,536, 1.86× at 131,072 (four threads). The ratio falls with size; a crossover above 262,144 columns, where CADO's distributed design pays off, is not excluded and was not reached here.

### 3.3. Pipeline against pipeline, before the port

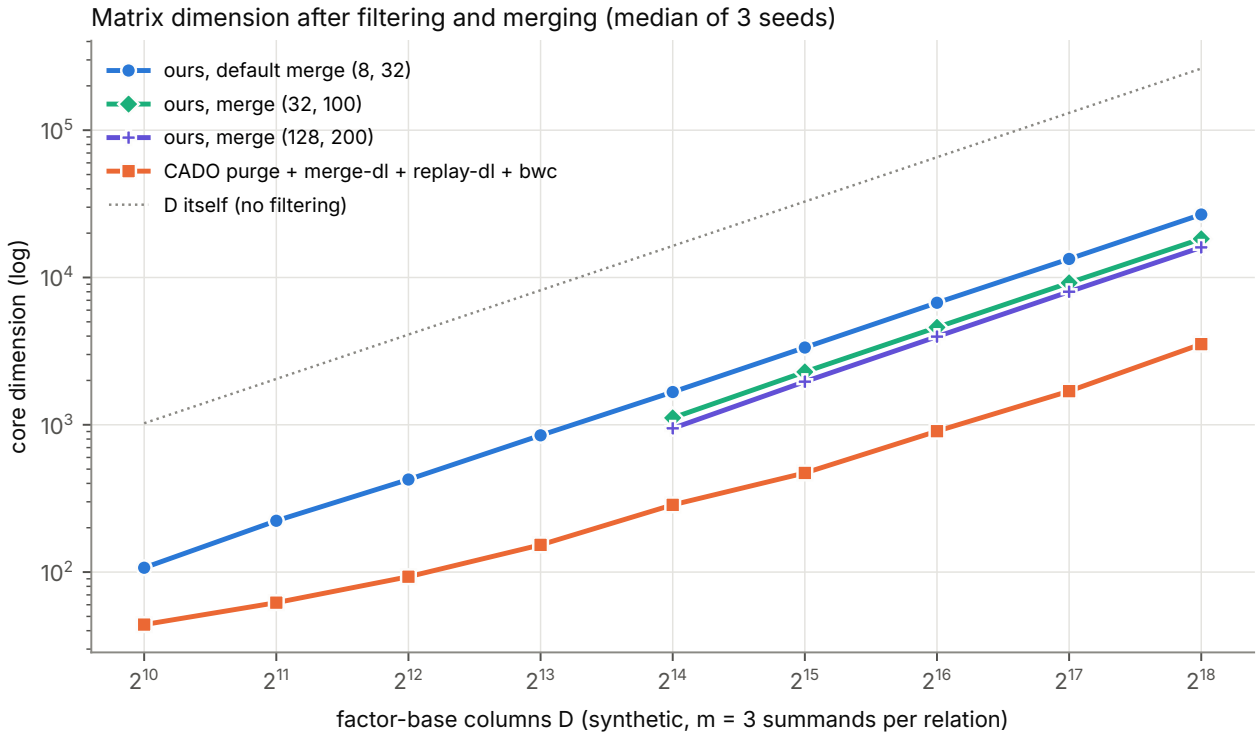



Figure 3: Matrix handed to the Krylov solver (amendments 1–2, +1 coefficients).

At D = 262,144 CADO-NFS's filter left 3,528 columns (257,515 nonzeros) and its pipeline took 15.0 s; ours left 26,756 and took 89.1 s. Raising our merge thresholds (32, 100) and (128, 200) gave 18,296 and 16,010 columns and 55.7 s and 61.9 s — the thresholds were not the gap. CADO-NFS's logs placed most of it in `purge`: after it, CADO kept 8,329 of 65,536 columns where our singleton + clique stage kept 32,860.

### 3.4. The port

`FilterStrategy::Cado` in `src/cryptanalysis/koblitz_sparse_la.rs`:

- **Purge** (CADO `purge`, Bouvier 2013): the excess is spent over up to 50 steps; each step deletes whole connected components of rows joined by weight-2 columns, heaviest first under $\Omega_{31}$ (a column of weight $w \geq 3$ contributes $0.8^{w-2}$, of weight 2 contributes 1/8), then cascades singletons.
- **Merge** (CADO `merge`): columns in order of the Markowitz estimate $(w-1)(r_{min}-2) - r_{min}$, each eliminated along a minimum spanning tree of its rows weighted by the exact size of each pairwise combination, until the average row weight passes the target density.
- `FilterOptions::cado()`: merge columns up to weight 32, no row cap, target density 100. `ic logs` uses it by default (`--filter-strategy legacy` restores the old filter). The library default stays the legacy filter and the new fields drop out of serialized options at their defaults, so frozen job configurations keep their digests and behaviour.
- Amendment 5 speeds the filter without changing its output (identical reports and cores on 12 checked cells): duplicate removal by parallel hashing (1.25 s ⟶ 0.08 s at D = 262,144), presized column lists (0.65 ⟶ 0.22 s), and a purge that computes component weights in parallel and sorts only the heaviest components each step.

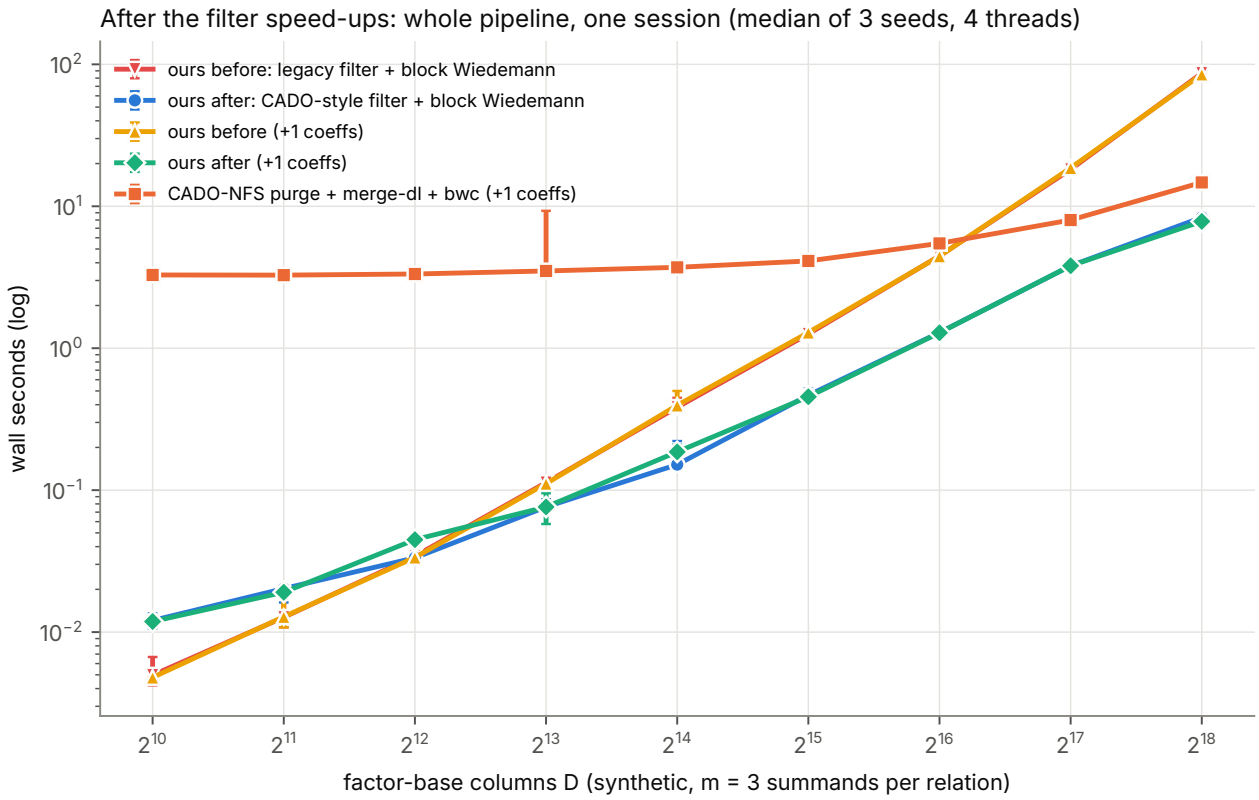



Figure 4: Amendment 5, one session: whole pipeline before and after, and CADO-NFS on the same +1-coefficient matrices. Median of 3 seeds, bars min–max.

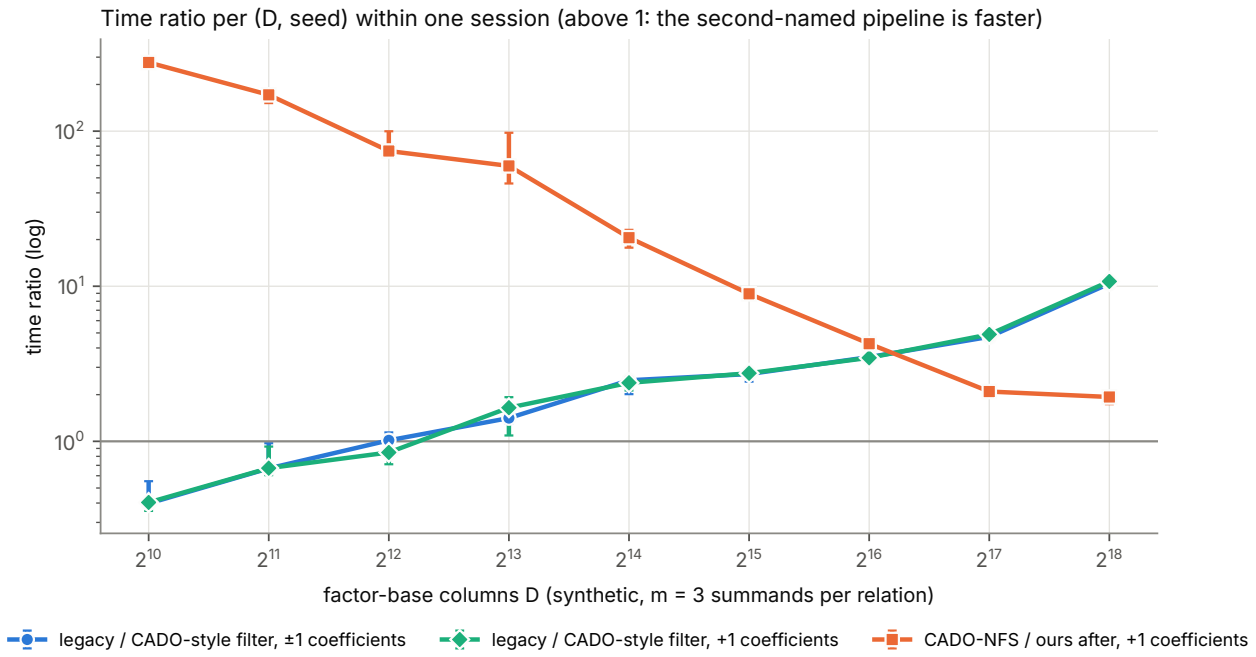



Figure 5: Per-(D, seed) time ratios in the amendment-5 session.

| D | legacy (±1) | CADO-style (±1) | legacy / CADO-style | CADO-NFS (+1) | CADO-NFS / ours (+1) |
|---|---|---|---|---|---|
| 16,384 | 0.38 s | 0.15 s | 2.48× [2.01, 2.53] | 3.71 s | 20.6× [17.7, 23.1] |
| 65,536 | 4.47 s | 1.29 s | 3.49× [3.46, 3.50] | 5.48 s | 4.26× [4.24, 4.50] |
| 131,072 | 18.19 s | 3.80 s | 4.72× [4.70, 5.07] | 8.00 s | 2.09× [2.09, 2.19] |
| 262,144 | 86.57 s | 8.35 s | 10.40× [9.99, 11.06] | 14.73 s | 1.93× [1.74, 1.96] |

Medians of 3 seeds; ratio intervals are the per-seed minimum and maximum. Cores at D = 262,144: legacy 26,750 columns, CADO-style 3,424 (±1) and 3,419 (+1), CADO-NFS 3,512. Amendment 4 (the port before the amendment-5 speed-ups) measured CADO-NFS / ours = 1.32× [1.14, 1.36] at 262,144; its target-density sweep at 262,144 gave 13.30 s (density 0), 10.75 s (50), 11.08 s (100, shipped) and 11.58 s (170).

## 3.5. End to end

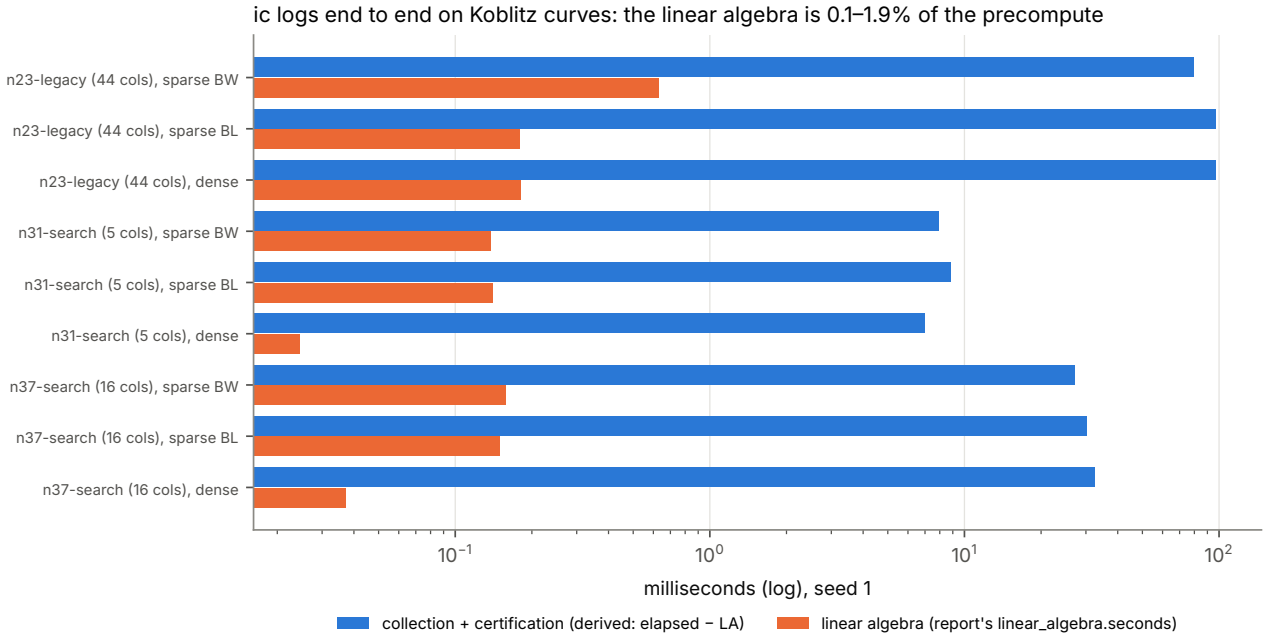



Figure 6: `ic logs` on Koblitz curves (n = 23 legacy base, n = 31 and 37 search-selected bases), seed 1; the non-linear-algebra time is the report's elapsed time minus `linear_algebra.seconds` (derived).

`ic logs` caps the factor-base dimension at 13 for materialization, so its real matrices have 5–44 columns here. **Derived projection, not measured:** at the N73 rate of 30.7 s of single-thread collection per relation (600 relations in 18,398.8 s), the 1,049,656 relations of the D = 262,144 synthetic system would take about 373 core-days to collect, against 8.35 s to solve. This assumes the N73 relation rate carries over, which a larger factor base would change (it raises the decomposition probability); it shows the scale, not a prediction.

## 4. Deviations, failures and limits

- Conductor's control plane could not start in this container (no Docker or Postgres); the work ran without a task lock.
- Three unmerged cores of the frozen grid (D = 8,192 seed 3, 16,384 seed 3, 32,768 seed 1) were singular after trimming to square; both engines returned row-consistent solutions that are not the planted vector, so those cells are unverified and excluded from medians (n = 2 there).
- The CADO-NFS pipeline at D = 1,024 seed 2 reconstructed all 1,024 logarithms but not the planted ones, in every session that ran it (amendments 1, 2, 4 and 5; `verified: false`), while our arms recovered the planted vector on the same system, so the system is uniquely solvable; the likely cause is a singular merged matrix after `merge-dl -target_excess 0`. The cell is excluded from CADO-NFS medians (n = 2 there).
- At m = 2 the unmerged core collapsed to 5–25 columns; CADO `bwc` exited 1 on the 25-column one, and the stop rule skipped its other seeds.
- CADO-NFS's `bwc` balancing aborted on some `merge-dl -t 4` outputs ("expects a column permutation replicated on rows"); amendment 2 rebalances with `mf_bal --reorder columns`, as the message asks, and records `reorder_retry`.
- The amendment-1 CADO arm first ran a stale binary (`raw/grid_amend1.falsestart.*`) and was restarted.
- Run-to-run drift on this host is up to about 1.4× for the same cell across sessions; every comparison above is within one session.
- One modulus width (62 bits) was measured; the 130-bit ECC2K-130 order is covered by tests, not timings. No GPU, no multi-node runs.

## 5. Validation

`cargo test --release --lib`: 4,092 passed, 0 failed, 117 ignored; `cargo test --release --bin ic`: 8 passed; `cargo clippy --locked --all-targets -- -D warnings` and `cargo fmt --all -- --check` clean. New tests: CADO-style filter on 60 random planted systems against the dense reference, the end-to-end sparse solve, both Krylov solvers modulo the 130-bit ECC2K-130 order, and the serialized-options digest.

## 6. References

C. Bouvier, *The filtering step of discrete logarithm and integer factorization algorithms*, 2013 (hal-00734654). S. Cavallar, *Strategies in filtering in the number field sieve*, ANTS-IV, 2000. D. Coppersmith, *Solving homogeneous linear equations over GF(2) via block Wiedemann algorithm*, Math. Comp. 62 (1994). CADO-NFS, `filter/purge.cpp`, `filter/clique_removal.cpp`, `filter/merge.cpp`, `linalg/bwc/` at commit `692ecb7e62f0f3bdab88ee44cc60b8ded0ea1a1b`.